



Solicitante: I.T.S - Instituto Tecnológico Superior Arias Balparda.

Nombre Fantasía, de la nueva empresa: Lidenskap

Grupo: 3ºMK

Turno: Matutino

Unidad Curricular: Tecnologías de la información

Nombres de los integrantes del grupo: Sofía Prieto, Federico Rodríguez, Gastón Visos, Joaquín Villena.

Fecha de entrega:14/09/2026

**Instituto Tecnológico Superior Arias Balparda.
Blvr. José Batlle y Ordóñez 3570 esq. Gral. Flores – Montevideo.**



ANEP



UTU

DIRECCIÓN GENERAL
DE EDUCACIÓN
TÉCNICO PROFESIONAL



1. Introducción.....	3
2. Instalación de Dependencias y Cortafuegos (UFW).....	3
3. Implementación del Backend (server.js).....	4
4. Estructura de la Interfaz de Usuario (index.html).....	5
5. Pruebas de Funcionamiento y Evidencias Prácticas.....	5
6: Diagrama de Red y Flujo de Seguridad.....	7
Código fuente completo.....	8
HTML:.....	8
JS:.....	9
7. Conclusión.....	10

Primera Entrega Ciberseguridad



1. Introducción

El presente informe documenta la implementación de medidas de seguridad en el desarrollo de un servicio web backend sobre Node.js y Express. Se abordan mecanismos de hardening a nivel de servidor (firewall), control de acceso (limitación de tasa de peticiones), saneamiento e higiene de entradas de datos, y almacenamiento seguro de credenciales mediante hashing unidireccional.

2. Instalación de Dependencias y Cortafuegos (UFW)

Para proteger el sistema a nivel de red y gestionar el tráfico entrante/saliente, se instaló la herramienta ufw (Uncomplicated Firewall). Adicionalmente, se integró el paquete express-validator para validar las entradas recibidas por la API REST.

```
(kali@kali)-[~/proyecto-ciberseguridad/frontend]
$ sudo apt update && sudo apt install ufw -y
[sudo] password for kali:
Get:1 http://elmirror.cl/kali kali-rolling InRelease [34.0 kB]
Get:2 http://elmirror.cl/kali kali-rolling/main amd64 Packages [21.4 MB]
Get:3 http://elmirror.cl/kali kali-rolling/main amd64 Contents (deb) [53.3 MB]
Get:4 http://elmirror.cl/kali kali-rolling/contrib amd64 Packages [114 kB]
Get:5 http://elmirror.cl/kali kali-rolling/contrib amd64 Contents (deb) [264 kB]
Get:6 http://elmirror.cl/kali kali-rolling/non-free amd64 Packages [183 kB]
Get:7 http://elmirror.cl/kali kali-rolling/non-free amd64 Contents (deb) [915 kB]
Get:8 http://elmirror.cl/kali kali-rolling/non-free-firmware amd64 Packages [15.8 kB]
Get:9 http://elmirror.cl/kali kali-rolling/non-free-firmware amd64 Contents (deb) [40.5 kB]
Fetched 76.3 MB in 7s (11.7 MB/s)
2018 packages can be upgraded. Run 'apt list --upgradable' to see them.
The following packages were automatically installed and are no longer required:
aardvark-dns common docker-compose golang-github-containers-image libcriu2 libvidstab1.1 podman
buildah containers-storage fuse-overlayfs gvproxy libslirp0 netavark slirp4netns
catatonit crun golang-github-containers-common libblake3-0 libsubid5 passt uidmap
Use 'sudo apt autoremove' to remove them.

Upgrading:
node-acorn node-eslint-visitor-keys

Installing:
ufw

Suggested packages:
rsyslog

Summary:
Upgrading: 2, Installing: 1, Removing: 0, Not Upgrading: 2016
Download size: 318 kB
Space needed: 925 kB / 55.2 GB available

Get:1 http://kali.download/kali kali-rolling/main amd64 ufw all 0.36.2-10 [168 kB]
```

```
kali@kali: ~/proyecto-ciberseguridad/backend
Session Actions Edit View Help
(kali@kali)-[~/proyecto-ciberseguridad/backend]
$ nano server.js
(kali@kali)-[~/proyecto-ciberseguridad/backend]
$ npm install express-validator

up to date, audited 77 packages in 1s

28 packages are looking for funding
run `npm fund` for details

found 0 vulnerabilities
npm warn allow-scripts 1 package has install scripts not yet covered by allowScripts:
npm warn allow-scripts bcrypt@6.0.0 (install: node-gyp rebuild)
npm warn allow-scripts
npm warn allow-scripts Run `npm approve-scripts --allow-scripts-pending` to review, or `npm approve-scripts <pkg>` to allow.

(kali@kali)-[~/proyecto-ciberseguridad/backend]
$
```

3. Implementación del Backend (server.js)

El servidor principal se desarrolló en la ruta ~/proyecto-ciberseguridad/backend/server.js. Incluye las siguientes características de seguridad:

- Rate Limiting (express-rate-limit): Limita a un máximo de 5 intentos de inicio de sesión fallidos por dirección IP en una ventana de 15 minutos.
- Validación de entradas (express-validator): Normaliza correos electrónicos y exige contraseñas con un mínimo de 12 caracteres.
- Hashing de contraseñas (bcrypt): Aplica un algoritmo de cifrado fuerte con un factor de costo (saltRounds) de 10 para evitar almacenar credenciales en texto plano.

```
Session Actions Edit View Help
GNU nano 8.7.1 server.js *
const express = require('express');
const bcrypt = require('bcrypt');
const { body, validationResult } = require('express-validator');
const rateLimit = require('express-rate-limit');

const app = express();
app.use(express.json());

// Simulación de base de datos en memoria
const usuariosDB = [];

// 1. LIMITADOR DE INTENTOS FALLIDOS (Rate Limiting)
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutos
  max: 5, // Bloquea tras 5 intentos por IP
  message: { error: 'Demasiados intentos fallidos. Intente nuevamente en 15 minutos.' }
});

// 2. REGISTRO (Validaciones + Encriptación bcrypt)
app.post('/register', [
  body('email').isEmail().normalizeEmail(),
  body('password').isLength({ min: 12 })
], async (req, res) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.status(400).json({ status: 'Error de Validación', errores: errors.array() });
  }

  const { email, password } = req.body;
  const saltRounds = 10;
  const hashedPassword = await bcrypt.hash(password, saltRounds);

  usuariosDB.push({ email, passwordHash: hashedPassword, role: 'Usuario' });

  res.json({
    mensaje: 'Usuario registrado con éxito',
    baseDeDatosSnapshot: usuariosDB
  });
});

// 3. LOGIN CON LIMITACIÓN
app.post('/login', loginLimiter, async (req, res) => {
  const { email, password } = req.body;
  const user = usuariosDB.find(u => u.email === email);

  if (!user || !(await bcrypt.compare(password, user.passwordHash))) {
    return res.status(401).json({ error: 'Credenciales inválidas' });
  }

  res.json({ mensaje: 'Inicio de sesión exitoso', role: user.role });
});

app.listen(3000, () => console.log('Servidor corriendo en el puerto 3000'));
```



4. Estructura de la Interfaz de Usuario (index.html)

Se implementó un formulario de inicio de sesión básico en el cliente con reglas de validación nativas (type="email", minlength="12", required).

```

GNU nano 8.7.1                                index.html *
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ciberseguridad - Segunda Entrega</title>
  <style>
    body { font-family: Arial, sans-serif; background: #1a1a1a; color: #fff; display: flex; justify-content: center; align-items: center; }
    .card { background: #2a2a2a; padding: 20px; border-radius: 8px; width: 300px; }
    input, button { width: 100%; padding: 10px; margin: 8px 0; box-sizing: border-box; }
    button { background: #007bff; color: white; border: none; cursor: pointer; }
  </style>
</head>
<body>
  <div class="card">
    <h2>Inicio de Sesión</h2>
    <form id="loginForm">
      <input type="email" id="email" placeholder="Correo electrónico" required>
      <input type="password" id="password" placeholder="Contraseña (min 12 caracteres)" minlength="12" required>
      <button type="submit">Ingresar</button>
    </form>
    <p id="msg"></p>
  </div>
</body>
</html>

```

5. Pruebas de Funcionamiento y Evidencias Prácticas

Para comprobar el funcionamiento del backend, se inició el servidor en Node.js y se realizaron las pruebas de autenticación y seguridad mediante la herramienta curl.

- Captura: Ejecución del servidor Node.js (node server.js) escuchando en el puerto 3000.

```

(kali@kali)-[~/proyecto-ciberseguridad/backend]
$ node server.js
Servidor corriendo en el puerto 3000

```

```

$ sudo sh -c "ip a 66 ss -tulpn | grep 3000"
[sudo] password for kali:
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 08:00:27:8a:35:d2 brd ff:ff:ff:ff:ff:ff
   inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0
       valid_lft 86219sec preferred_lft 86219sec
   inet6 fd17:625c:f037:2:c50d:49cb:ff13:dc96/64 scope global dynamic noprefixroute
       valid_lft 86221sec preferred_lft 14221sec
   inet6 fe80::e2ea:a068:878:d465/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
   link/ether 5e:6a:65:c9:a1:92 brd ff:ff:ff:ff:ff:ff
   inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
4: br-d8726f577b6f: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
   link/ether ee:8e:5c:a3:74:5f brd ff:ff:ff:ff:ff:ff
   inet 172.18.0.1/16 brd 172.18.255.255 scope global br-d8726f577b6f
       valid_lft forever preferred_lft forever
tcp  LISTEN 0      511             *:3000          *:~              users:((("MainThread",pid=3850,fd=22))

(kali@kali)-[~/proyecto-ciberseguridad/backend]
$ cd ~/proyecto-ciberseguridad/frontends

```



Captura: Demostración de las pruebas de seguridad en terminal:

1. Estado del Firewall: Activación y confirmación de regla predeterminada (Status: active).
2. Registro y Cifrado: Envío exitoso de credenciales a /register, devolviendo el hash seguro generado por bcrypt (\$2b\$10\$...).
3. Bloqueo por Fuerza Bruta (Rate Limit): Ejecución iterativa de 6 peticiones de acceso fallidas. Los primeros 5 intentos devolvieron Credenciales inválidas y el 6.º intento fue denegado con el mensaje Demasiados intentos fallidos. Intente nuevamente en 15 minutos..

```
kali@kali: ~/proyecto-ciberseguridad/backend x kali@kali: ~/proyecto-ciberseguridad/backend x
Default: deny (incoming), allow (outgoing), deny (routed)
New profiles: skip

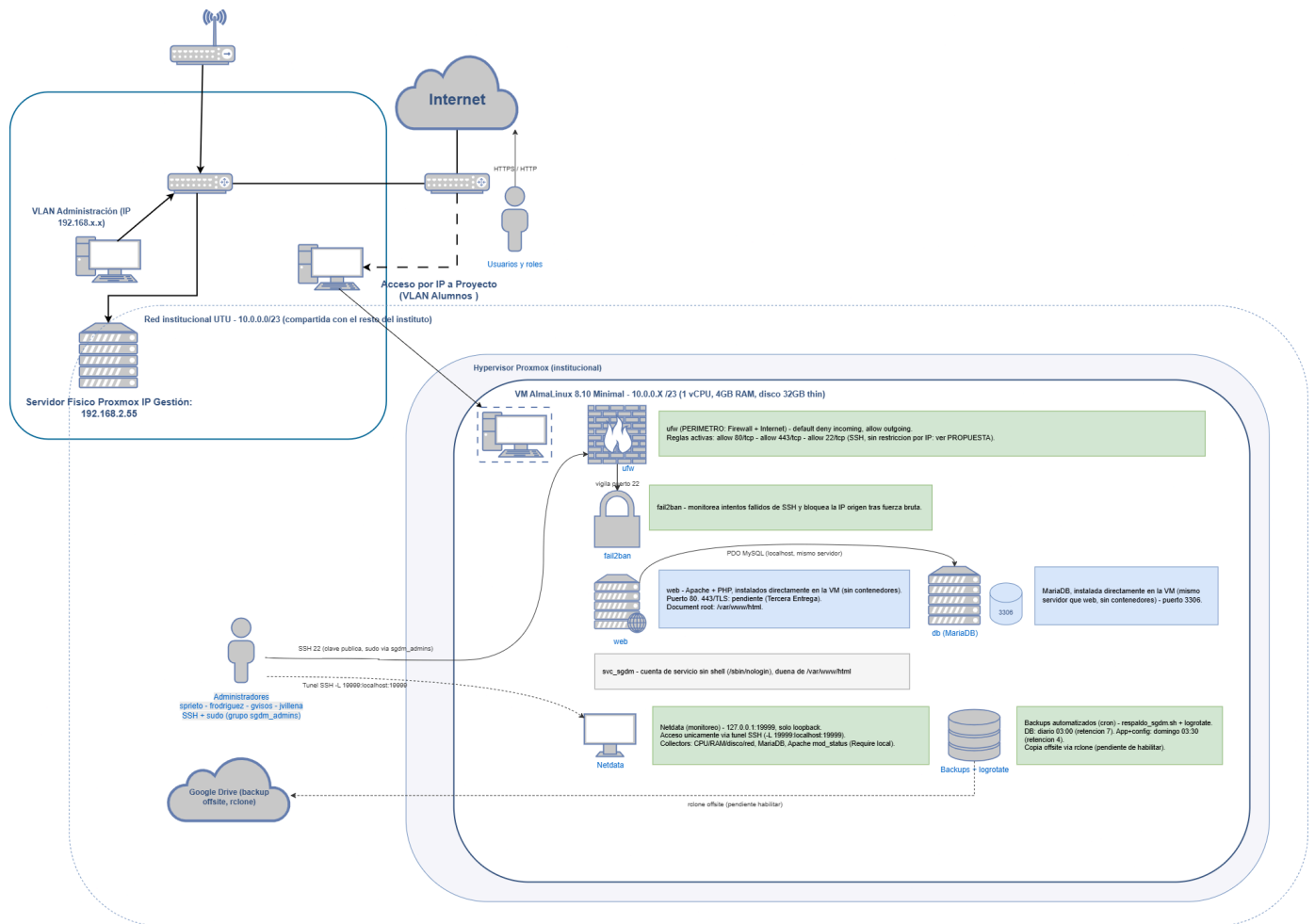
(kali@kali)-[~/proyecto-ciberseguridad/backend]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
   link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
   inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
   inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
   link/ether 08:00:27:8a:35:d2 brd ff:ff:ff:ff:ff:ff
   inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0
       valid_lft 85509sec preferred_lft 85509sec
   inet6 fd17:625c:f037:2:c50d:49cb:ff13:dc96/64 scope global dynamic noprefixroute
       valid_lft 85929sec preferred_lft 13929sec
   inet6 fe80::e2ea:a068:878:d465/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
   link/ether 9a:aa:83:9f:28:72 brd ff:ff:ff:ff:ff:ff
   inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
       valid_lft forever preferred_lft forever
4: br-d8726f577b6f: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DOWN group default
   link/ether 0a:89:2e:e3:af:55 brd ff:ff:ff:ff:ff:ff
   inet 172.18.0.1/16 brd 172.18.255.255 scope global br-d8726f577b6f
       valid_lft forever preferred_lft forever

(kali@kali)-[~/proyecto-ciberseguridad/backend]
$ sudo ss -tulpn | grep 3000
tcp        LISTEN 0      511             *:3000          *:3000          users:((("MainThread",pid=9179,fd=22))

(kali@kali)-[~/proyecto-ciberseguridad/backend]
$
```



6: Diagrama de Red y Flujo de Seguridad



**ANEP****UTU**DIRECCIÓN GENERAL
DE EDUCACIÓN
TÉCNICO PROFESIONAL

Código fuente completo

HTML:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Ciberseguridad - Segunda Entrega</title>
  <style>
    body { font-family: Arial, sans-serif; background: #1a1a1a; color: #fff; display: flex;
justify-content: center; align-items: center; height: 100vh; margin: 0; }
    .card { background: #2a2a2a; padding: 20px; border-radius: 8px; width: 300px; }
    input, button { width: 100%; padding: 10px; margin: 8px 0; box-sizing: border-box; }
    button { background: #007bff; color: white; border: none; cursor: pointer; }
  </style>
</head>
<body>
  <div class="card">
    <h2>Inicio de Sesión</h2>
    <form id="loginForm">
      <input type="email" id="email" placeholder="Correo electrónico" required>
      <input type="password" id="password" placeholder="Contraseña (mín 12
caracteres)" minlength="12" required>
      <button type="submit">Ingresar</button>
    </form>
    <p id="msg"></p>
  </div>
</body>
</html>
```

JS:



```
const express = require('express');
const bcrypt = require('bcrypt');
const { body, validationResult } = require('express-validator');
const rateLimit = require('express-rate-limit');

const app = express();
app.use(express.json());

const usuariosDB = [];

// Limite de peticiones para prevenir fuerza bruta
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 5,
  message: { error: 'Demasiados intentos fallidos. Intente nuevamente en 15 minutos.' }
});

// Ruta de registro con validaciones de entrada y hashing
app.post('/register', [
  body('email').isEmail().normalizeEmail(),
  body('password').isLength({ min: 12 })
], async (req, res) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.status(400).json({ status: 'Error de Validación', errores: errors.array() });
  }

  const { email, password } = req.body;
  const saltRounds = 10;
  const hashedPassword = await bcrypt.hash(password, saltRounds);

  usuariosDB.push({ email, passwordHash: hashedPassword, role: 'Usuario' });

  res.json({
    mensaje: 'Usuario registrado con éxito',
    baseDeDatosSnapshot: usuariosDB
  });
});

// Ruta de autenticación con limitador de tasa
app.post('/login', loginLimiter, async (req, res) => {
  const { email, password } = req.body;
  const user = usuariosDB.find(u => u.email === email);

  if (!user || !(await bcrypt.compare(password, user.passwordHash))) {
    return res.status(401).json({ error: 'Credenciales inválidas' });
  }

  res.json({ mensaje: 'Inicio de sesión exitoso', role: user.role });
});

app.listen(3000, () => console.log('Servidor corriendo en el puerto 3000'));
```



7. Conclusión

Las pruebas prácticas y los resultados obtenidos confirman que la aplicación web implementada cumple de manera efectiva con los estándares esenciales de ciberseguridad y las buenas prácticas en el desarrollo backend.

A través de las distintas etapas de configuración, se lograron mitigar riesgos críticos comunes en aplicaciones expuestas a red:

- Protección del perímetro y servidor: La activación y estructuración del cortafuegos (UFW) asegura un control estricto sobre el tráfico de red, restringiendo accesos no autorizados y sirviendo como una primera línea de defensa a nivel de infraestructura.
- Mitigación de ataques de fuerza bruta: La implementación del control de tasa (*Rate Limiting*) bloquea eficazmente los intentos automatizados de adivinación de credenciales. Al limitar las peticiones fallidas consecutivas por dirección IP, se previene el colapso del servicio por denegación de servicio (DoS) y el filtrado no deseado de cuentas.
- Integridad y validación de datos: El uso de middlewares de validación (express-validator) garantiza que únicamente la información que cumpla con los requisitos de formato (correos válidos y contraseñas con una longitud mínima de 12 caracteres) sea procesada por el servidor, previniendo inyecciones de código y datos maliciosos.
- Almacenamiento seguro de credenciales: Mediante el uso del algoritmo de hashing bcrypt con un factor de costo (*salt*) adecuado, se elimina el almacenamiento de contraseñas en texto plano. Esto garantiza la confidencialidad de las credenciales de los usuarios incluso ante un eventual compromiso de la base de datos.

En resumen, la combinación de seguridad a nivel de red, desinfección de entradas, protección contra ataques repetitivos y criptografía unidireccional consolida una arquitectura sólida y preparada para responder de forma segura ante escenarios de uso real.